
答卷编号（参赛学校填写）：

答卷编号（竞赛组委会填写）：

论文题目：**基于提高打孔机生产效能的数学模型**

组 别：**本科生**

参赛学校：长春理工大学

报名序号：（可以不填）

参赛队员信息（必填）：

	姓 名	专业班级及学号	联系电话
参赛队员 1	陈洋	应用物理学 100122226	18704497184
参赛队员 2	朱保琨	电子信息工程 090411335	15543118624
参赛队员 3	金钊	应用物理学 100122220	18704497311

答卷编号（竞赛组委会填写）：

评阅情况（省赛评阅专家填写）：

省赛评阅 1：

省赛评阅 2：

省赛评阅 3：

省赛评阅 4：

省赛评阅 5：

摘 要

基于过孔是印刷线路板（也称为印刷电路板）的重要组成部分之一，过孔的加工费用通常占制板费用的 30%到 40%，打孔机主要用于在制造印刷线路板流程中的打孔作业。欲提高打孔机的生产效率可通过缩短钻头的加工路径长度来降低钻头移动时间，路径的优化程度是印刷电路板打孔机性能的重要指标。对其优化利于缩短打孔机的作业、刀具转换、钻头行进时间以提高生产效能。首先对刀具路径进行建模，应用蚁群算法对刀具路径进行求解，然后引入双钻头对孔群同时加工，并对双钻头的孔群加工优化路径进行数学建模，采用遗传算法得到基于单钻头的孔群加工优化路径。选择单钻头对环境进行建模简单，根据单钻头模型得出打孔机工作最优路线，且行进时间等于作业最短距离除以钻头的行进速度再减去刀具转换总时间，根据钻头行进总成本和刀具转换总成本，可得单钻头作业成本。再则，利用分析单钻头的行进时间和作业成本方法，得出双钻头的在最优作业路线条件下的行进时间和作业成本，并与传统单钻头打孔机进行比较，同时考虑打孔机的两钻头的合作间距对作业路线和生产效能产生的影响。实验结果表明，双钻头最优加工路径与单钻头的最优加工路径相比，在不同钻孔速度下使用双钻头同时加工的新算法能节省加工时间，有效提高了打孔机的加工质量、加工效率、生产效能。

关键词：生产效能 蚁群算法 遗传算法 路径优化

1 问题重述

(1) 由某块印刷线路板过孔中心坐标的数据，单位是密尔 (mil) (也称为毫英寸，1 inch=1000 mil)，给出单钻头作业的最优作业线路 (包括刀具转换方案)、行进时间和作业成本。

(2) 为提高打孔机效能，设计一种双钻头的打孔机 (每个钻头的形状与单钻头相同)，两钻头可以同时作业，且作业是独立的，即可以两个钻头同时进行打孔，也可以一个钻头打孔，另一个钻头行进或转换刀具。为避免钻头间的触碰和干扰，在过孔加工的任何时刻必须保持两钻头间距不小于 3cm (称为两钻头合作间距)。为使问题简化，可以将钻头看作质点。

(i) 针对坐标数据，给出双钻头作业时的最优作业线路、行进时间和作业成本，并与传统单钻头打孔机进行比较，其生产效能提高多少。

(ii) 研究打孔机的两钻头合作间距对作业路线和生产效能产生的影响。

2 问题分析

本题旨在解决如何提高某类打孔机的生产效能。所谓生产效能，即单位时间内的生产能力、加工效率。欲提高打孔机的生产效率可通过缩短钻头的加工路径长度来降低钻头移动时间。孔群加工路径优化问题为典型的旅行商问题，简称为 TSP，对于单钻头的孔群路径优化问题多利用蚁群算法进行计算，双钻头加工中的两条路径去掉对刀点合并后也是一个待加工孔的序列，因此通过类似的染色体表示双钻头孔群加工路径问题的可能解，即利用遗传算法进行解析。根据数学知识先求单钻头问题，行进时间等于作业最短距离除以钻头的行进速度再减去刀具转换总时间，根据钻头行进总成本和刀具转换总成本，可得单钻头作业成本。当使用双钻头作业时，为使问题简化，可以将钻头看作质点，双钻头的两钻头可以同时作业，且作业是独立的，即可以两个钻头同时进行打孔，也可以一个钻头打孔，另一个钻头行进或转换刀具，为避免钻头间的触碰和干扰，现假设在过孔加工的任何时刻必须保持两钻头间距等于 3cm (称为两钻头合作间距)，利用上述遗传算法做出双钻头最优作业路线图，可知双钻头最短工作路线，可利用分析单钻头的行进时间和作业成本方法得出双钻头的在最优作业路线条件下的行进时间和作业成本并与传统单钻头打孔机进行比较，可知其生产效能提高了多少。最后研究打孔机的两钻头的合作间距对作业路线和生产效能产生的影响。

3 模型设计

- 1) 假定对于同一孔型钻孔作业时间都是相同的；
- 2) 假定钻头的行进速度是相同的；
- 3) 假定在过孔加工的任何时刻必须保持两钻头间距等于 3cm；
- 4) 假定将钻头看做质点。
- 5) 假设用数控钻铣床进行孔加工时，单钻头、双钻头加工过程如下：

单钻头：对某一给定尺寸的孔，抑止对应的刀具后，从下刀点开始，沿着使该刀具总的窄程最短的轨迹，从一个孔移动到另一个孔，直到该类孔中的所有对象都被加工完毕，再进行下一尺寸的也孔的加工，如此循环。把问题描述成以下优化模型：

(1) 变量设计。设有 n 个孔的集合 $V_1, V_2, \dots, V_n$ 设 D_{ij} , 表示集合中任意两孔之间的距离。

(2) 目标函数。需要在孔集合 V 中, 找到一个不重复的全排列 $T = \{T_1, T_2, \dots, T_n\}$, 令

$$M = \sum_{i,j>0 \text{ 且 } i \neq j}^{n-1} d(T_i, T_j), \text{ 求 } M \text{ 的最小值。}$$

(3) 约束条件。加工路径从一个孔出发, 对每一个孔只加工一次遍历每一个孔, 最后回到起点, 包括刀具转换在内。

(4) 优化算法, 蚁群算法。

双钻头: 双钻头在孔群加工中, 两个钻头同时加工, 每个钻头加工时间并不确定, 那么单一工件的加工时间由耗时较长的钻头确定。

假设两个钻头的对刀点分别为 S_1 和 S_2 , A_{1i} 和 A_{2j} 分别表示第一个钻头加工的第 i 个孔和第 2 个钻头加工的第 j 个孔, 则两条加工路径 U_1 、 U_2 分别为

$$\{S_1 - A_{11} - \dots - A_{1i} - \dots - S_1\} \quad \{S_2 - A_{21} - \dots - A_{2j} - \dots - S_2\}$$

双钻头孔群加工路径优化的目标函数为:

$$\min \quad T = \begin{cases} T_1 & T_1 > T_2 \\ T_2 & T_1 \leq T_2 \end{cases} \quad (0) \text{ 式}$$

相应的加工时间分别为 T_1 和 T_2 。

约束条件: 任意一个待加工的孔必须包括在其中一条加工路径中, 且加工过程中两个钻头不发生碰撞。

4 算法的理论分析

4.1 蚁群算法

4.1.1 蚁群算法简述

蚁群算法本质上是一种随机搜索算法, 它是通过对候选解组成的群体的进化来寻求最优解。算法由许多蚂蚁共同完成, 每只蚂蚁在候选解的空间中独立搜索解, 并在所寻得的解上留下一定的信息素。解的性能越好, 蚂蚁留在其上的信息素就越多, 信息素越多的解被选择的可能性也越大。在算法的最初阶段所有解上的信息素是相同的, 随着算法的推进, 较优解上的信息素将逐渐增多, 算法渐渐趋于收敛。

引用 M. Dorigo 所举的例子, 来说明蚁群的运动情况。如图 1 所示, 设 A 是巢穴, E 是食物源, HC 为一障碍物。假设每个时间单位有 30 只蚂蚁由 A 到达有, 也有 30 只蚂蚁从 E

到达 A。初始时刻，蚂蚁以相同的概率选择每条路。经过一段时间后，在路径 BCD 上的信息素是 BHD 上的 2 倍，将有 20 只蚂蚁由 BCD 到达 E。随着时间推移，蚂蚁将会以越来越大的概率选择路径 BCD，从而找到最短路径。

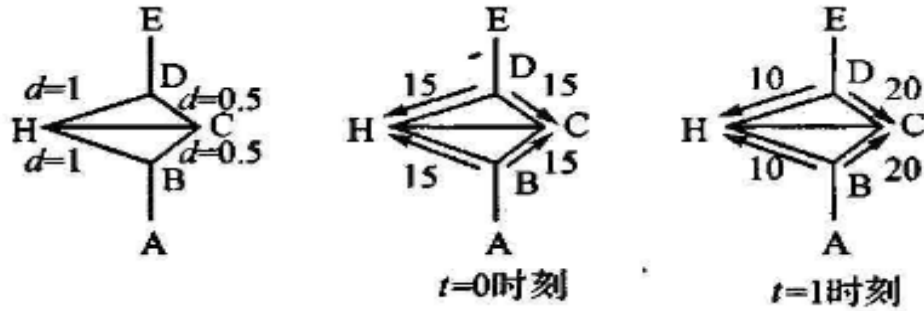


图 1 蚂蚁觅食实验路线图

4.1.2 算法实现

(1) 状态转移规则

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ik}(t)]^\beta}{\sum [\tau_{is}(t)]^\alpha \cdot [\eta_{is}(t)]^\beta}, & \text{若 } j \in Allowed_k, s \in allowed_k \\ 0, & \text{else} \end{cases} \quad (1) \text{ 式}$$

式中 $p_{ij}^k(t)$ ——在 t 时刻蚂蚁 k 由元素 i 转移到元素 j 的概率； $Allowed_k$ ——表示蚂蚁 k 下一步允许选择的的城市； α ——信息启发式因子，表示轨迹的相对重要性； β ——期望启发式因子，表示能见度的相对重要性； $\eta_{ij}(t)$ ——启发函数， $\eta_{ij}(t) = 1/d_{ij}$ ； τ_{ij} ——残留信息量。

(2) 局部调整规则：局部调整是每只蚂蚁在建立一个解的过程中进行的，经过 h 个时刻，两个元素状态之间的局部信息素数量要根据下式作调整：

$$\tau_{ij}(t+h) = (1-\zeta)\tau_{ij}(t) + \zeta\tau_0$$

$$\tau_0 = 1/(nl_{\min})$$

式中： $\zeta \in [0,1]$ ， $l_{\min}$ 表示所有坐标集合中两个最近元素之间的距离。

(3) 全局调整准则：只有生成了全局最优解的蚂蚁才有机会进行全局调整，全局调整规则为

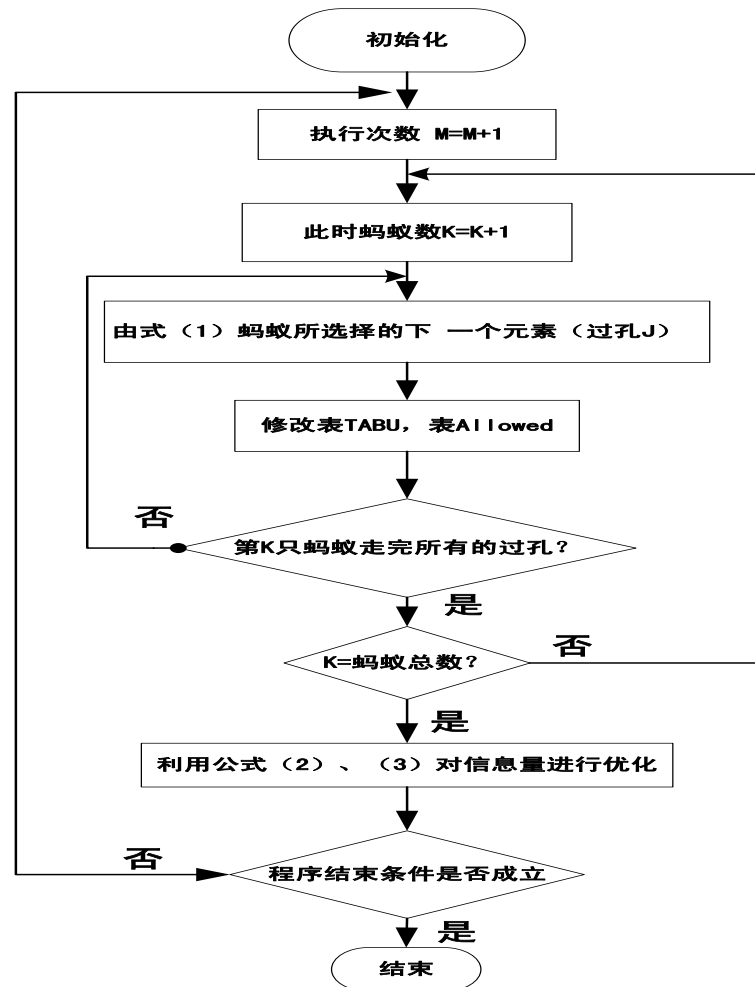
$$\begin{aligned}\tau_{ij}(t+n) &= (1+\rho) \times \tau_{ij}(t) + \Delta\tau_{ij}(t) \\ \Delta\tau_{ij}(t) &= \sum_k^m \Delta\tau_{ij}^k(t) \\ \Delta\tau_{ij}^k(t) &= \begin{cases} \frac{Q}{L_k}, & \text{若第}k\text{只在本次循环中}(i,j) \\ 0, & \text{else} \end{cases} \quad (2)(3) \text{ 式}\end{aligned}$$

式中 ρ ——信息素挥发系数； $\Delta\tau_{ij}^k(t)$ ——表示第 k 只蚂蚁在本次循环中留在路径 (i, j) 上的信息量； Q ——信息素强度，设为常数； L_k ——第 k 只蚂蚁在本次循环中所走的路径的长度。

本文编写蚁群算法程序的步骤如下：

- 1) 初始化问题的集合规模 n , 蚂蚁的数量 m , 并将 m 只蚂蚁放到 n 个城市（过孔）上；
- 2) 程序执行需要循环 $M_c = M_c + 1$ 次；
- 3) 执行循环时蚂蚁的个数 $K = K + 1$ ；
- 4) 对其中第 K 只蚂蚁，根据公式（1）选择城市（过孔） j , 并继续进行前进；
- 5) 把蚂蚁选择的城市（过孔） j 加入到第 k 只蚂蚁的表 tabu 中，并修改表 Allowed ；
- 6) 对于第 K 只蚂蚁如果没有行走完所有 n 个城市（过孔），则转到第四步，若所有城市（过孔）均已走完，则继续往下执行；
- 7) 如果蚂蚁数 K 小于蚂蚁总数 m , 则转到第三步，直到 m 只蚂蚁都走完 n 个城市，继续往下执行；
- 8) 由式（2）、式（3）随时更新蚂蚁行走时的信息量，且找出 m 只蚂蚁中，所走路径最短的值，并保存；
- 9) 若循环的总次数没有达到最大的循环次数，则将继续转到第二步进行执行，若满足结束条件循环将结束，同时输出结果数据。

程序流程图如下：



4.2 遗传算法

4.2.1 遗传算法简述

遗传算法 (GA) 是一种优化技术, 其目的是提高解决问题能力, 保持最佳组合输入变量。从代表问题潜在解集的一个种群开始的, 而一个种群则由经过基因编码的一定数目的个体组成。每个个体实际上是染色体带有特征的实体。染色体作为遗传物质的主要载体, 即多个基因的集合, 其基因型是某种基因组合决定的。因此, 在一开始需要实现从表现型到基因型的映射即编码工作。由于仿照基因编码的工作很复杂, 往往对其进行简化, 如采用二进制编码。原始种群产生之后, 按照适者生存和优胜劣汰的原理, 逐代演化产生出越来越好的近似解。在每一代, 根据问题域中个体的适应度大小挑选个体, 并借助于自然遗传学的遗传算子进行组合交叉和变异, 产生出代表新解集的种群。这个过程将导致种群像自然进化一样, 其后代种群比前代更加适应于环境, 末代种群中的最优个体经过解码, 可以作为问题近似最优解, 遗传算法对于组合优化中的 NP 问题非常有效。

4.2.2 算法实现

遗传算法在优化孔群加工路径中，染色体一般为一个待加工孔的序列，所以染色体长度与孔的数量相等。直接采用孔的编号编码在运算中可能出现某些孔未加工的情况，因此可采用编码方式如下：每加工一个孔就将其从未加工列表 T 中删除，并将该孔在 T 中的位置序号放入顺序列表 R 中，依次进行同样操作直到 T 中所有孔被删除，则列表 R 作为一个染色体表示一个待加工孔的序列。假设某个待加工孔的序列为：

$A_1 - A_2 - A_4 - A_3 - A_8 - A_5 - A_9 - A_6 - A_7$ ，则按上述方法编码得到的染色体为 (112141311)。

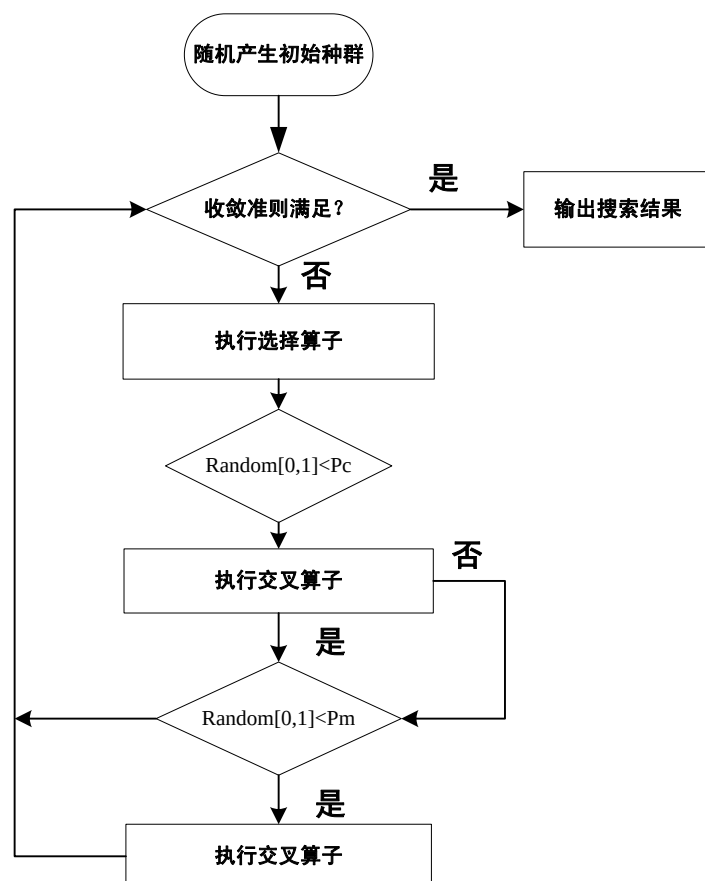
而对于双钻头孔群加工路径问题，每个孔的加工序列都是两条加工路径去掉对刀点后合并的结果，所以还需要把这个序列分为两个子序列。序列中每两个相邻孔之间断开后都可形成两个子序列，根据钻头行走时间和钻孔所需时间可算出对应加工时间为 T_1 和 T_2 。

由前面 (0) 式可求得对应的加工时间 T ，比较不同断点对应的 T 值， T 值最小的两个子序列就是这一染色体代表的双钻头孔群加工方案。在遗传算法中，适应度值是判断个体是否优良和 解是否较优的依据。孔群加工最优路径是以加工时间最短为目标函数，因此，对于任意不发生碰撞的双钻头加工路径 U_1 、 U_2 对应顺序列表 R 所表示染色体的适应度为：

$$F(T) = 1/T \quad (4) \text{式}$$

对于发生碰撞的双钻头加工路径 U_1 、 U_2 适应度为 0。

程序流程图如下：



5 模型运行结果的分析

5.1 问题一结果分析

对于单钻头的假设，工作时采用蚁群算法由 MATLAB 进行数据处理，得出路径最优图形、平均距离和最短距离分别如图二、如图三；

且由图二、图三可知在单钻头作业的最优作业线路（包括刀具转换方案）的条件下，可以得到作业最短距离为：

$$\text{Shortest_Length} = 4.7023\text{e}+008(\text{mil})$$

所有钻头的行进速度是相同的，为 180mm/s，且所有钻头的刀具转换时间为：

$$36*270+18*212+3*18*95+18*64+4*18*20+2*18*10+3*18*29=23184(\text{s})$$

所以，行进时间=作业总时间-钻头转换时间；作业总时间=最短距离/钻头行进速度
即：

$$4.7023\text{e}+008(\text{mil}) = 1.1943842\text{e}+007(\text{mm})$$

$$\text{作业总时间} = 1.1943842\text{e}+007(\text{mm}) / 180\text{mm/s} = 6.6354\text{e}+004(\text{s})$$

$$\text{行进时间} = 6.6354\text{e}+004(\text{s}) - 23184(\text{s}) = 43170.7(\text{s})$$

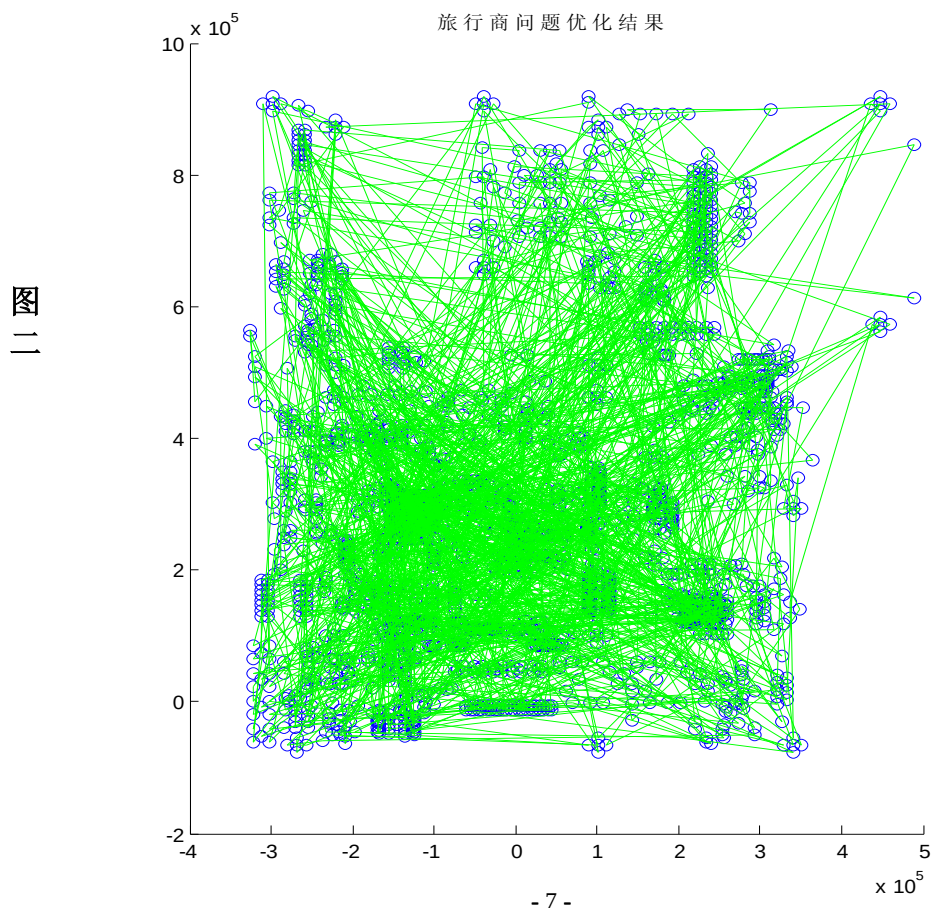
综上，作业成本=行进成本+刀具转换时间成本

$$\text{即：行进成本} = 1.1943842\text{e}+007(\text{mm}) * 0.06 \text{元/mm} = 7.166\text{e}+005(\text{元})$$

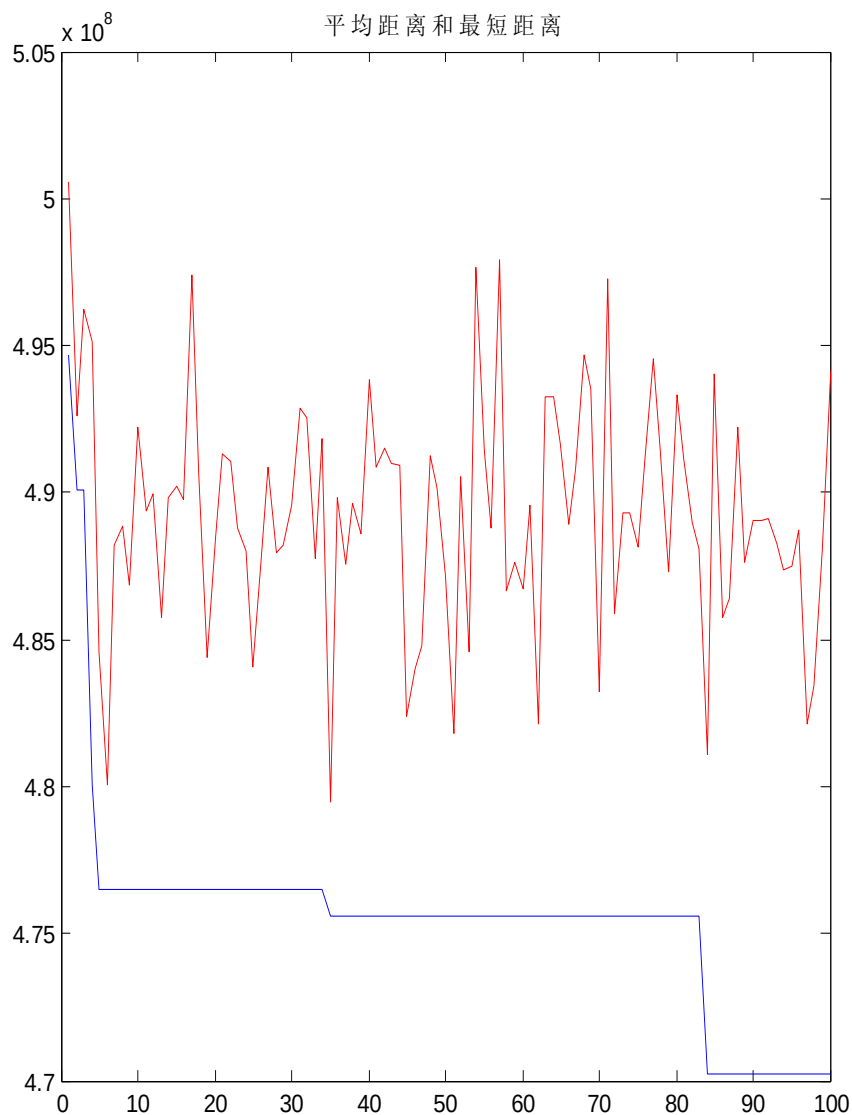
$$\text{刀具转换时间成本} = 23184(\text{s}) * 7(\text{元/min}) / 60 = 2704.8(\text{元})$$

$$\text{作业成本} = 7.166\text{e}+005(\text{元}) + 2704.8(\text{元}) = 7.1933\text{e}+005(\text{元})$$

路径最优图形



平均距离和最短距离图形



图三

5.2 问题二结果分析

同理，由双钻头的假设，利用遗传算法对双钻头进行计算，结果如图四，其中包括：

(i) 根据假定可知，双钻头过孔加工的任何时刻保证两钻头间距为 3cm，则由最优作业路线图可知双钻头作业最短为：

$$\text{Shortest_Length} = 4.6790\text{e}+008(\text{mil})$$

钻头的行进速度依旧为 180mm/s，刀具转换时间与单钻头相同为 23184s，同理，行进时间=作业总时间—钻头转换时间，作业总时间=最短距离/钻头行进速度

即：

$$4.6790\text{e}+008(\text{mil})=1.1884660\text{e}+007(\text{mm})$$

$$\text{作业总时间}=1.1884660\text{e}+007(\text{mm})/180(\text{mm/s})=6.6026\text{e}+004(\text{s})$$

$$\text{行进时间}=6.6026\text{e}+004(\text{s})-23184(\text{s})=4.2842\text{e}+004(\text{s})$$

同理：作业成本=行进成本+刀具转换时间成本

$$\text{行进成本}=1.1884660\text{e}+007(\text{mm})\times 0.06\text{元/mm}=7.131\text{e}+004(\text{元})$$

$$\text{刀具转换时间成本}=23184(\text{s})\times 7(\text{元/min})/60/2=1352.4(\text{元})$$

$$\text{作业成本}=7.131\text{e}+004(\text{元})+1352.4(\text{元})=7.2692\text{e}+004(\text{元})$$

双钻头与传统单钻头进行比较：

$$(1)\text{孔机缩短的作业时间}=6.6354\text{e}+004(\text{s})-6.6026\text{e}+004(\text{s})=328(\text{s})$$

$$(2)\text{刀具转换缩短的时间}=23184(\text{s})/2=11592(\text{s})$$

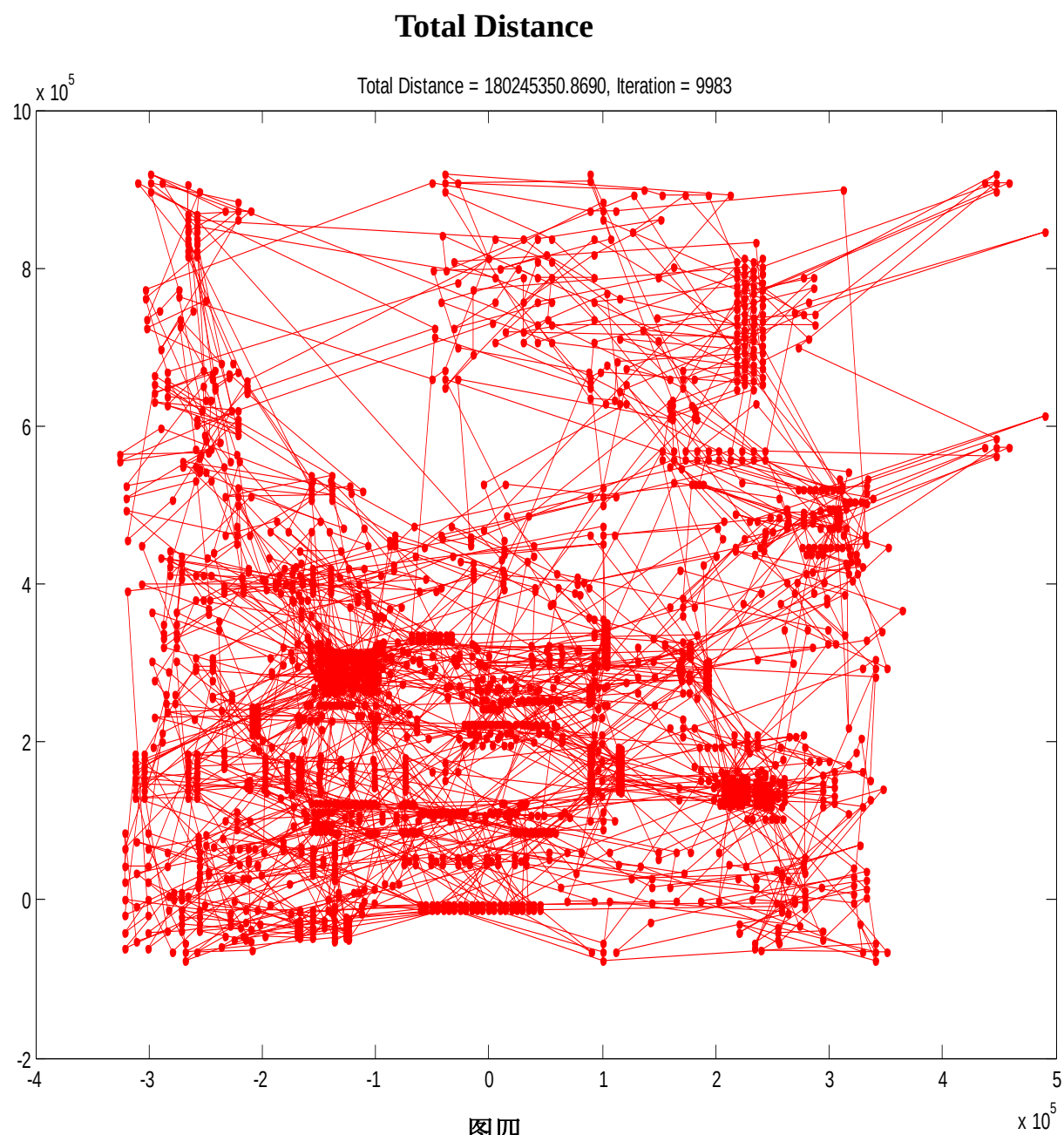
$$(3)\text{钻头行进缩短的时间}=43170.7(\text{s})-4.2842\text{e}+004(\text{s})=328.7(\text{s})$$

由（1）~（3）可知，双钻头打孔机与传统单钻头打孔机进行比较，其生产效能确有提高。

（ii）研究打孔机的两钻头合作间距对作业路线和生产效能产生的影响

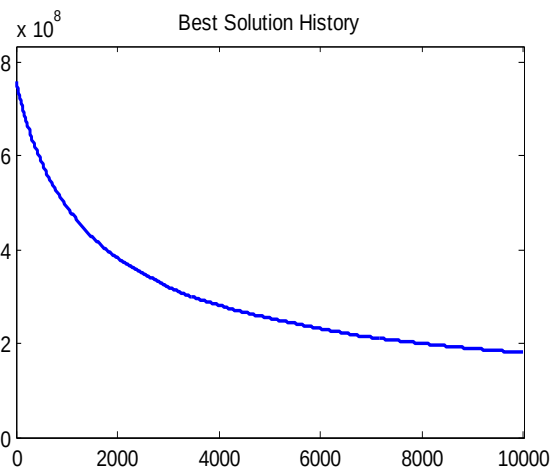
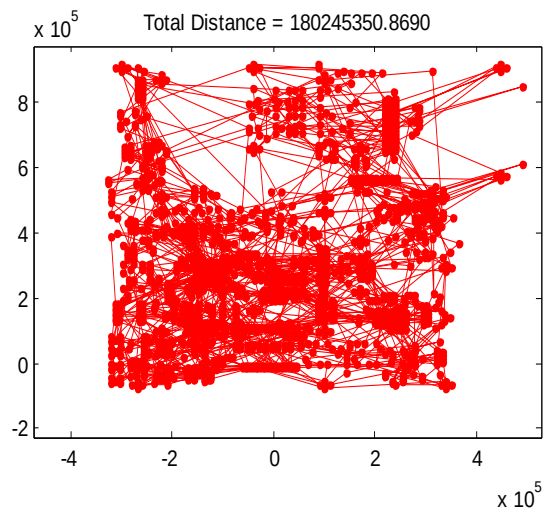
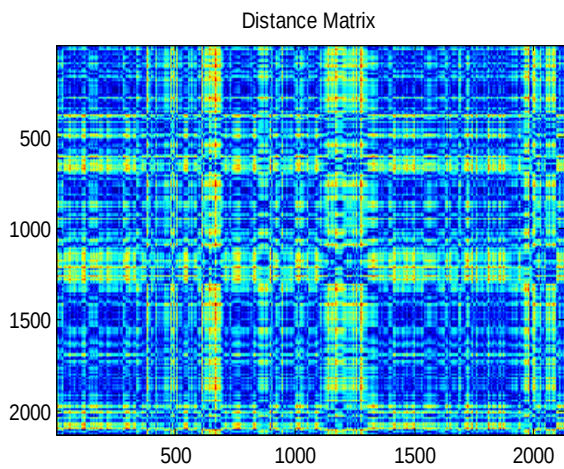
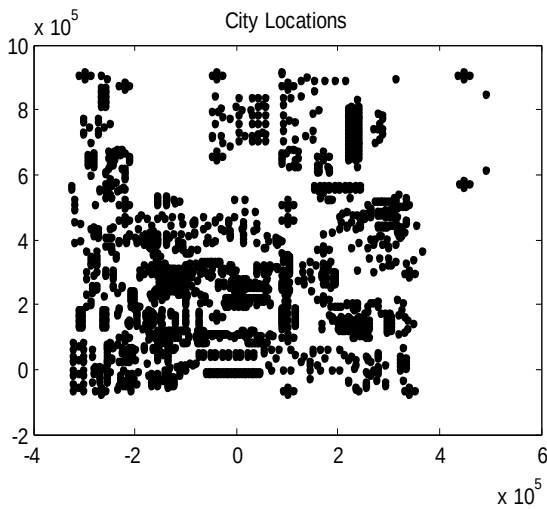
由题意可知，打孔机的双钻头合作间距不小于 3 厘米，在保证此要求的前提下，还需缩短打孔机的作业、刀具转换的、钻头行进时间，以提高生产效能及优化作业路线，则需选择一个合适的打孔机的两钻头合作间距，以得到最优路径，进而减少行进成本以及作业成本。由于，打孔机的双钻头合作间距会影响作业路线，合作间距控制不合理将会浪费资源，即而会导致两个钻头不同时作业，使加工效率、加工质量降低，以使整体工作时间加长、产品质量下降，且作业总时间与作业成本有直接联系，作业成本与生产效能亦息息相关。综合上述的作业成本与作业时间的关系式，间距的合理性将会对整个系统起到至关重要的作用。则必须选择最合理的合作间距以提高工作效率、工作质量、打孔速度、缩短时间成本，即而缩短作业成本，以达到提高生产效能目的。路径的最优是缩短钻头的加工路径长度来降低钻头移动时间，其与间距的合理设置密切相关，综上所述，生产效能的提高归根结底是合作间距的合理性。

最优路线与其辅助图形如下：

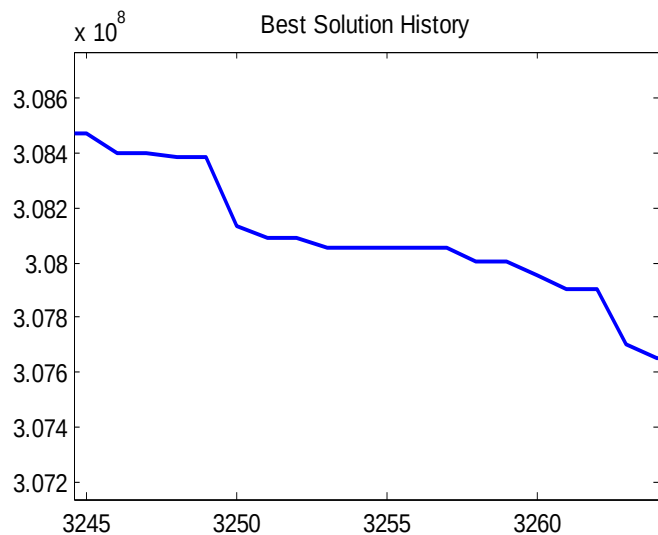
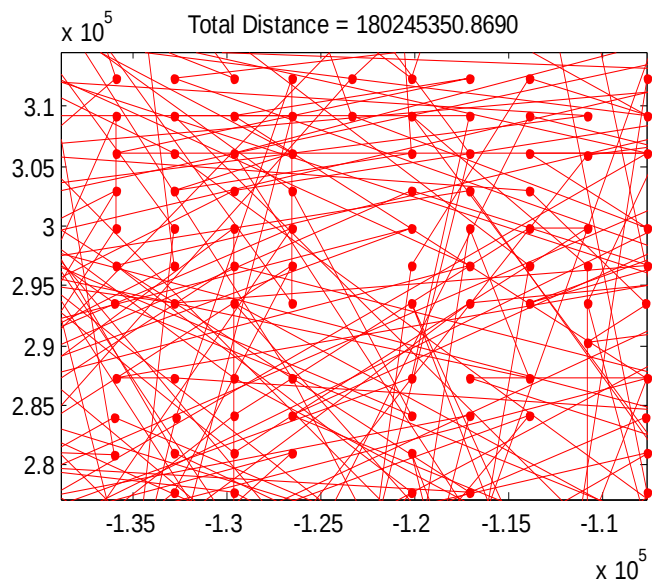


图四

City Locations、Distance Matrix、Total Distance、 Best Solution History



Total Distance、 Best Solution History 图形部分放大图形



6 模型评价与改进

模型的核心思路是在第一问建立模型基础上, 解决第二问对于双钻头的求解方法, 将最终解决了提高打孔机工作效能的问题, 该模型分析过程清晰、简单易行, 且通过实验验证了双钻头提高了工作效能且节省刀具的转换、钻头的行进、钻孔作业的时间。引入双钻头同时加工是提高孔群加工效率的有效途径, 分析双钻头孔群加工路径优化问题的目标函数和约束条件后, 修改针对单钻头孔群加工路径优化的遗传算法得到用于双钻头的孔群加工路径优化算法。本文只通过一组试验数据分析得出结果, 会有误差, 所以在此基础上, 用多组数据分析, 会更好的减小误差, 也可以对做模型进行进检验。但实验结果仍然表明新算法在钻头不同钻孔速度下其加工时间较大程度缩短, 而由此较大幅度的提高了孔群加工的效率, 即而生产效能得到提高。

参考文献:

- [1]肖人彬, 陶振武. 孔群加工路径规划问题的进化求解 [J]. 计算机集成制造系统, 2005, 11(5): 682—689
- [2]王霄. PCB 数控钻孔最佳走刀路线的建模与求解[J]. 计算机辅助设计与图形学学报, 2001, 13(7): 590. 593
- [3]姜昌华, 胡幼华. 一种求解旅行商问题的高效混合遗传算法[J]. 计算机工程与应用, 2004. 40(22): 67. 70
- [4]Hong R B, Jin F H, Gao Q J. Multilayer feedfor2 ward neural network based on ant colony system[J]. J of Harbin Institute of Technology, 2003, 35(7): 8232-825.
- [5]丁建立, 陈增强, 袁著祉. 基于自适应蚂蚁算法的动态最优路由选择[J]. 控制与决策, 2003, 18(6): 7512-753
- [6]田贵超, 黎明, 韦雪洁. 旅行商问题 (TSP) 的几种求解方法[J]. 计算机仿真, 2006. 23 (8): 153-157.
- [7]李擎, 张伟, 尹怡欣, 等. 一种用于最优路径规划的改进遗传算法[J]. 信息与控制, 2006, 35 (4): 444-447.
- [8]周明, 孙树栋. 遗传算法原理及应用[M]. 北京: 国防工业出版社, 1999
- [9]段海滨. 蚁群算法原理及其应用[M]. 北京: 科学出版社, 2005

附录:

单钻头于蚁群算法 MATLAB 计算部分程序如下:

%%第二步: 将 m 只蚂蚁放到 n 个城市 (过孔) 上

```

Randpos=[]; %随即存取
for i=1:(ceil(m/n))
    Randpos=[Randpos, randperm(n)];
end
Tabu(:,1)=(Randpos(1,1:m))';
%%第三步:  $m$  只蚂蚁按概率函数选择下一座城市 (过孔), 完成各自的周游
for j=2:n %所在城市 (过孔) 不计算
    for i=1:m
        visited=Tabu(i,1:(j-1)); %记录已访问的城市 (过孔), 避免重复访问
        J=zeros(1, (n-j+1)); %待访问的城市 (过孔)
        P=J; %待访问城市 (过孔) 的选择概率分布
        Jc=1;
        for k=1:n
            if length(find(visited==k))==0 %开始时置 0
                J(Jc)=k;
                Jc=Jc+1; %访问的城市 (过孔) 个数自加 1
            end
        end
        %下面计算待选城市 (过孔) 的概率分布
        for k=1:length(J)
            P(k)=(Tau(visited(end), J(k)) ^ Alpha) * (Eta(visited(end), J(k)) ^ Beta);
        end
        P=P/(sum(P));
        %按概率原则选取下一个城市 (过孔)
        Pcum=cumsum(P); %cumsum, 元素累加即求和
        Select=find(Pcum>=rand); %若计算的概率大于原来的就选择这条路线
        to_visit=J(Select(1));
        Tabu(i,j)=to_visit;
    end
end
if NC>=2
    Tabu(1,:)=R_best(NC-1,:);
end
%%第四步: 记录本次迭代最佳路线
L=zeros(m,1); %开始距离为 0,  $m \times 1$  的列向量
for i=1:m
    R=Tabu(i,:);
    for j=1:(n-1)
```

```

        L(i)=L(i)+D(R(j),R(j+1));    %原距离加上第 j 个城市（过孔）到第 j+1 个城市（过孔）的距离
    end
    L(i)=L(i)+D(R(1),R(n));          %一轮下来后走过的距离
    end
    L_best(NC)=min(L);                %最佳距离取最小
    pos=find(L==L_best(NC));
    R_best(NC,:)=Tabu(pos(1),:); %此轮迭代后的最佳路线
    L_ave(NC)=mean(L);                %此轮迭代后的平均距离
    NC=NC+1                           %迭代继续
    %%第五步：更新信息素
    Delta_Tau=zeros(n,n);             %开始时信息素为 n*n 的 0 矩阵
    for i=1:m
        for j=1:(n-1)

Delta_Tau(Tabu(i,j),Tabu(i,j+1))=Delta_Tau(Tabu(i,j),Tabu(i,j+1))+Q/L(i);

        %此次循环在路径（i，j）上的信息素增量
        end
        Delta_Tau(Tabu(i,n),Tabu(i,1))=Delta_Tau(Tabu(i,n),Tabu(i,1))+Q/L(i);
        %此次循环在整个路径上的信息素增量
        end
        Tau=(1-Rho).*Tau+Delta_Tau; %考虑信息素挥发，更新后的信息素
        %%第六步：禁忌表清零
        Tabu=zeros(m,n);              %%直到最大迭代次数
        end
        %%第七步：输出结果
        Pos=find(L_best==min(L_best)); %找到最佳路径（非 0 为真）
        Shortest_Route=R_best(Pos(1),:)%最大迭代次数后最佳路径
        Shortest_Length=L_best(Pos(1)) %最大迭代次数后最短距离
        subplot(1,2,1)                 %绘制第一个子图形
        DrawRoute(C,Shortest_Route)    %画路线图的子函数
        subplot(1,2,2)                 %绘制第二个子图形
        plot(L_best)
        hold on                         %保持图形
        plot(L_ave,'r')
        title('平均距离和最短距离')   %标题
    function DrawRoute(C,R)

```

双钻头于遗传算法 MATLAB 计算部分程序如下：

```

% Verify Inputs
[N,dims] = size(xy);
[nr,nc] = size(dmat);

```

```

if N ~= nr || N ~= nc
    error('Invalid XY or DMAT inputs!')
end
n = N;
% Sanity Checks
pop_size = 4*ceil(pop_size/4);
num_iter = max(1,round(real(num_iter(1))));
show_prog = logical(show_prog(1));
show_res = logical(show_res(1));
% Initialize
pop = zeros(pop_size,n);
for k = 1:pop_size
    pop(k,:) = randperm(n);
end
% Run the GA
global_min = Inf;
total_dist = zeros(1,pop_size);
dist_history = zeros(1,num_iter);
tmp_pop = zeros(4,n);
new_pop = zeros(pop_size,n);
if show_prog
    pfig = figure('Name','TSP_GA | Current Best
Solution','Numbertitle','off');
end
for iter = 1:num_iter
    % Evaluate Each Population Member (Calculate Total Distance)
    for p = 1:pop_size
        d = dmat(pop(p,n),pop(p,1)); % Closed Path
        for k = 2:n
            d = d + dmat(pop(p,k-1),pop(p,k));
        end
        total_dist(p) = d;
    end
    % Find the Best Route
    [min_dist,index] = min(total_dist);
    dist_history(iter) = min_dist;
    if min_dist < global_min
        global_min = min_dist;
        opt_rte = pop(index,:);
        if show_prog
            % Plot the Best Route
            figure(pfig);
            rte = opt_rte([1:n 1]);
            if dims == 3, plot3(xy(rte,1),xy(rte,2),xy(rte,3),'r.-');

```

```

        else plot(xy(rte,1),xy(rte,2),'r.-'); end
        title(sprintf('Total      Distance      =      %1.4f,      Iteration
= %d',min_dist,iter));
    end
end
% Genetic Algorithm Operators
rand_pair = randperm(pop_size);
for p = 4:4:pop_size
    rtes = pop(rand_pair(p-3:p),:);
    dists = total_dist(rand_pair(p-3:p));
    [ignore,idx] = min(dists);
    best_of_4_rte = rtes(idx,:);
    ins_pts = sort(ceil(n*rand(1,2)));
    I = ins_pts(1);
    J = ins_pts(2);
    for k = 1:4 % Mutate the Best to get Three New Routes
        tmp_pop(k,:) = best_of_4_rte;
        switch k
            case 2 % Flip
                tmp_pop(k,I:J) = fliplr(tmp_pop(k,I:J));
            case 3 % Swap
                tmp_pop(k,[I J]) = tmp_pop(k,[J I]);
            case 4 % Slide
                tmp_pop(k,I:J) = tmp_pop(k,[I+1:J I]);
            otherwise % Do Nothing
        end
    end
    new_pop(p-3:p,:) = tmp_pop;
end
pop = new_pop;
end

```